Experiment No : 8

Title:Implement Binary search tree with operations Create, search, and recursive traversal.

Name of Student:Shivam Sevewar
Class: E&TC Div: SY1 PRN No:B25ET1042 Batch:B

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left, *right;
};

struct node* createNode(int value) {
   struct node *newNode = malloc(sizeof(struct node));

   if (newNode == NULL) {
      printf("Memory allocation failed!\n");
      exit(1);
   }

   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}

struct node* insert(struct node *root, int value) {
   if (root == NULL)
      return createNode(value);

   if (value < root->data)
      root->left = insert(root->left, value);
```

```
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

struct node* search(struct node *root, int key) {
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);

    return search(root->right, key);
}

void inorder(struct node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
```

```
        printf("%d ", root->data);
    }
}

int main() {
    struct node *root = NULL;
    int choice, value, key, n, i;

    do {
        printf("\n--- BINARY SEARCH TREE ---\n");
        printf("1. Create BST\n");
        printf("2. Search an element\n");
        printf("3. Inorder Traversal\n");
        printf("4. Preorder Traversal\n");
        printf("5. Postorder Traversal\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("How many nodes? ");
                scanf("%d", &n);

                for (i = 0; i < n; i++) {
                    printf("Enter value: ");
                    scanf("%d", &value);
                    root = insert(root, value);
                }
                printf("BST created successfully!\n");
                break;

            case 2:
                printf("Enter element to search: ");
                scanf("%d", &key);
```

```
            if (search(root, key) != NULL)
                printf("Element found!\n");
            else
                printf("Element not found!\n");
            break;

        case 3:
            printf("Inorder: ");
            inorder(root);
            printf("\n");
            break;

        case 4:
            printf("Preorder: ");
            preorder(root);
            printf("\n");
            break;

        case 5:
            printf("Postorder: ");
            postorder(root);
            printf("\n");
            break;

        case 6:
            printf("Program terminated.\n");
            break;

        default:
            printf("Invalid choice!\n");
      }
   } while (choice != 6);

   return 0;
}
```

Output:

--- BINARY SEARCH TREE ---
1. Create BST
2. Search an element
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit

Enter your choice: 1
How many nodes? 5
Enter value: 50
Enter value: 30
Enter value: 70
Enter value: 20
Enter value: 40
BST created successfully!

Enter your choice: 3
Inorder: 20 30 40 50 70

Enter your choice: 4
Preorder: 50 30 20 40 70

Enter your choice: 5
Postorder: 20 40 30 70 50

Enter your choice: 2
Enter element to search: 40
Element found!

Enter your choice: 6
Program terminated.